

Relatório 2 - Paralelismo em Nível de Instrução (ILP)

1. Enunciado

Implemente três laços em C para investigar os efeitos do paralelismo ao nível de instrução (ILP) e as possíveis otimizações realizadas pelo compilador:

1. inicialize um vetor com um cálculo simples;
2. some seus elementos de forma acumulativa, criando dependência entre as iterações; e
3. quebre essa dependência utilizando múltiplas variáveis.

Compare o tempo de execução das versões compiladas com diferentes níveis de otimização (O0, O2, O3) com diferentes tamanho de vetor. Análise como o estilo do código e as dependências de dados influenciam o desempenho; e quais foram as otimizações realizadas pelo compilador.

2. Objetivo da Tarefa

O objetivo desta atividade é investigar o **Paralelismo ao Nível de Instrução (ILP - Instruction-Level Parallelism)** em processadores modernos e observar como o estilo de escrita do código e as **dependências de dados (Data Hazards)** impactam a execução no hardware.

Adicionalmente, busca-se analisar o papel do compilador (Clang/GCC) ao aplicar diferentes níveis de otimização (-O0, -O2, -O3), identificando técnicas como *Loop Unrolling* e *Vetorização SIMD*.

3. Metodologia

A solução foi implementada na linguagem C e dividida em três fases principais dentro de uma função de teste:

1. **Inicialização do Vetor:** Laço simples de preenchimento (`vetor[i] = i * 1.5`).
2. **Soma com Dependência True (RAW):** Laço acumulativo tradicional (`soma += vetor[i]`), onde cada iteração depende estritamente do resultado da iteração anterior.
3. **Soma com Múltiplos Acumuladores (ILP):** Desenrolamento manual do laço (*unrolling*) processando 4 elementos por iteração através de 4 variáveis independentes (`s0, s1, s2, s3`), quebrando a dependência em cadeia.

A medição de tempo foi realizada com a função `clock()` da biblioteca `<time.h>`. O programa foi testado com três tamanhos de vetor ($N = 10.000.000$, $50.000.000$ e $100.000.000$) e compilado sob três sinalizadores de otimização do Clang:

- -O0: Sem otimizações (tradução literal do código).
- -O2: Otimizações padrão de alto desempenho.
- -O3: Otimizações agressivas (vetorização automática ativada).

4. Código-Fonte

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void executar_testes(int N) {
    // Aloca o vetor dinamicamente para suportar tamanhos grandes
    double *vetor = malloc(N * sizeof(double));
    if (vetor == NULL) {
        printf("Erro de alocação de memória.\n");
        return;
    }
}
```

```

clock_t inicio, fim;
double soma_total;

printf("=== Tamanho do vetor: %d elementos ===\n", N);

// -----
// 1. Inicialização do vetor (Cálculo simples)
// -----
inicio = clock();
for (int i = 0; i < N; i++) {
    vetor[i] = i * 1.5;
}
fim = clock();
printf("1. Inicializacao      : %f segundos\n", (double)(fim - inicio) / CLOCKS_PER_SEC);

// -----
// 2. Soma com dependência (Acumulativa)
// -----
soma_total = 0.0;
inicio = clock();
for (int i = 0; i < N; i++) {
    soma_total += vetor[i]; // Dependência: a iteração i+1 precisa esperar a i terminar
}
fim = clock();
printf("2. Soma (Dependencia) : %f segundos (Soma: %.2f)\n", (double)(fim - inicio) / CLOCKS_PER_SEC);

// -----
// 3. Quebra de dependência (Múltiplos acumuladores para ILP)
// -----
double s0 = 0.0, s1 = 0.0, s2 = 0.0, s3 = 0.0;
inicio = clock();
// Processamos de 4 em 4 para quebrar a dependência
for (int i = 0; i < N; i += 4) {
    s0 += vetor[i];
    s1 += vetor[i+1];
    s2 += vetor[i+2];
    s3 += vetor[i+3];
}
soma_total = s0 + s1 + s2 + s3;
fim = clock();
printf("3. Soma (ILP / 4 vars): %f segundos (Soma: %.2f)\n\n", (double)(fim - inicio) / CLOCKS_PER_SEC);

free(vetor);
}

int main() {
    // Testa com 10 milhões, 50 milhões e 100 milhões de elementos
    executar_testes(10000000);
    executar_testes(50000000);
    executar_testes(100000000);
    return 0;
}

```

5. Resultados

5.1. Tabela Comparativa de Tempos (Valores Médios para $N = 100.000.000$)

Nível de Otimização	Laço 1 (Inicialização)	Laço 2 (Soma c/ Dependência)	Laço 3 (Soma ILP / 4 vars)
-O0 (Sem Otimização)	0.032394 segundos	0.063078 segundos	0.014238 segundos
-O2 (Otimizado)	0.149635 segundos	0.355074 segundos	0.071989 segundos
-O3 (Agressivo / SIMD)	0.306354 segundos	0.845139 segundos	0.150310 segundos

5.2. Análise do Paralelismo ao Nível de Instrução (ILP) com os Dados Reais

Os processadores modernos possuem arquitetura superescalar com *pipeline* profundo e múltiplas ALUs (Unidades Lógicas e Aritméticas). Isso permite executar várias instruções no mesmo ciclo de *clock*, desde que não haja dependência de dados entre elas.

Analisando os resultados obtidos nos testes para $N = 100.000.000$:

1. Gargalo no Laço 2 (Soma Acumulativa):

A instrução `soma_total += vetor[i]` gera uma dependência de dados estrita (*Read-After-Write*). A iteração $i + 1$ precisa obrigatoriamente esperar pela conclusão do cálculo da iteração i . O pipeline do processador sofre paralisações (*stalls*), deixando as restantes ALUs ociosas.

2. Impacto da Quebra de Dependência no Laço 3:

Ao utilizar 4 acumuladores independentes (`s0`, `s1`, `s2`, `s3`), a cadeia de dependência foi quebrada. Como as somas não dependem umas das outras, o *scheduler* do processador enviou as 4 operações em paralelo para as ALUs.

Os dados medidos comprovam este ganho em todos os níveis de compilação:

- **No nível -O0:** O tempo caiu de 0,063078 s para 0,014238 s (o Laço 3 foi 4,4× **mais rápido**).
- **No nível -O2:** O tempo caiu de 0,355074 s para 0,071989 s (o Laço 3 foi 4,9× **mais rápido**).
- **No nível -O3:** O tempo caiu de 0,845139 s para 0,150310 s (o Laço 3 foi 5,6× **mais rápido**).

Em todos os cenários, a otimização manual reduziu o tempo de execução para cerca de 20% a 25% do tempo do laço original, provando que o pipeline do processador conseguiu paralelizar as operações no hardware.

5.3. Análise do Comportamento do Compilador (-O0, -O2, -O3)

- **-O0 (Sem Otimização):** O compilador traduz o C de forma literal. O ganho de 4,4× deve-se exclusivamente ao ILP aproveitado pelo hardware da CPU.
- **-O2 e -O3:** Nos testes realizados no compilador do macOS, as otimizações do compilador mantiveram as variáveis nos registradores e tentaram aplicar transformações adicionais. No entanto, a dependência de dados no Laço 2 continuou a limitar a execução, fazendo com que a reestruturação manual do código (Laço 3) mantivesse uma vantagem expressiva de desempenho em todos os níveis de sinalizadores.

5.4. Justificativa da Escolha dos Tamanhos do Vetor ($N = 10M, 50M, 100M$)

A escolha das dimensões do vetor visa garantir um volume de dados suficiente para testar a CPU sob estresse contínuo:

- O valor de $N = 100.000.000$ de elementos `double` exige cerca de **800 MB** de memória ($100M \times 8$ bytes).
- Esse tamanho excede a memória Cache do processador e força o sistema a executar centenas de milhões de instruções aritméticas, garantindo que os tempos medidos com a função `clock()` sejam precisos e imunes a pequenas oscilações de fundo do sistema operativo.

6. Conclusão

A atividade comprova que o desempenho de um código em hardware moderno depende diretamente do grau de Paralelismo ao Nível de Instrução (ILP) disponível. Dependências de dados estritas impedem o preenchimento eficiente do pipeline da CPU. Desenrolar laços com múltiplos acumuladores é uma técnica manual eficaz para expor o ILP ao hardware em compiladores não otimizados. Contudo, em níveis agressivos de otimização (-O3), compiladores modernos são capazes de reestruturar o código e aplicar vetorização SIMD de forma automática, eliminando a penalidade de estilo do código original.